

Creating and Labeling Data

Samuel Rowe adapted from Mitchell (2020)

2026-08-30

Contents

Overview	5
1 Creating Data	7
1.1 Creating and changing variables	7
1.2 Numeric expressions and functions	13
1.3 Strings	15
2 Labeling Data	25
2.1 Describing datasets	25
2.2 Labeling variables	33
2.3 Labeling values	35
2.4 Labeling utilities	44
2.5 Labeling variables and values in different languages	48
2.6 Using Notes	48
3 Exercises	51
3.1 Current Population Survey	51

bookdown::render_book()

Overview

Topics:

1. Creating Data
2. Labeling Data
3. Exercises

Chapter 1

Creating Data

The *generate* and *replace* will be the workhorses of creating and modifying variables. We'll deal with generating binary variables. We'll look into missing data and how to handle it. *Note: Please do not label your missing data, it just is a pain to deal with later* The *egen* command is also a very practical and useful (I wish other software packages had this type of command). We'll deal with strings and converting between numerics and strings.

1.1 Creating and changing variables

One of the most basic tasks is to create, replace, and modify variables. Stata provides some nice and easy commands to work with variable generation and modification relative to other statistical software packages: *generate* and *replace*. These will be the workhorses for the creation and modification of variables.

As we go through Mitchell's Chapter 6, we'll see that these are not the only two commands we will utilize. For example, Stata has a very helpful command called *egen*. It can be used to create new variables from old variables, especially when working with groups of observations. *Egen* becomes even more helpful when it is combined with *bysort*. We can sort our group and find the max, min, sum, etc. of a group of observations, which can come in handy when we are working with panel data. Furthermore, using indexes can make our generation commands more useful for creating lags or rebasing our data.

1.1.1 Generate command

The *generate* or *gen* command is one command that you may already have plenty of experience with, but there are some helpful tips when working with *generate*.

What we'll do first is to create a new variable called potential experience and potential experience squared.

Let's look at ages for workers earning weekly wages.

```
cd "/Users/Sam/Desktop/Data/CPS"
use smalljul25pub, clear
summarize prtage if prrelg==1
generate pot_exp=prtage-16
summarize pot_exp if prrelg==1
```

/Users/Sam/Desktop/Data/CPS

Variable	Obs	Mean	Std. dev.	Min	Max
prtage	9,635	42.43778	14.74111	15	85

(28,617 missing values generated)

Variable	Obs	Mean	Std. dev.	Min	Max
pot_exp	9,635	26.43778	14.74111	-1	69

Look for outliers and generate a histogram for outliers

```
histogram pot_exp if prrelg==1
graph export "jul_25_pot_exp.png", replace
```

Now let's create a new variable for potential experience squared. We have two options:

```
gen pot_exp2 = pot_exp*pot_exp
sum pot_exp2
drop pot_exp2

gen pot_exp2 = pot_exp^2
sum pot_exp2
```

(28,617 missing values generated)

Variable	Obs	Mean	Std. dev.	Min	Max
pot_exp2	93,635	1269.006	1345.314	0	4761

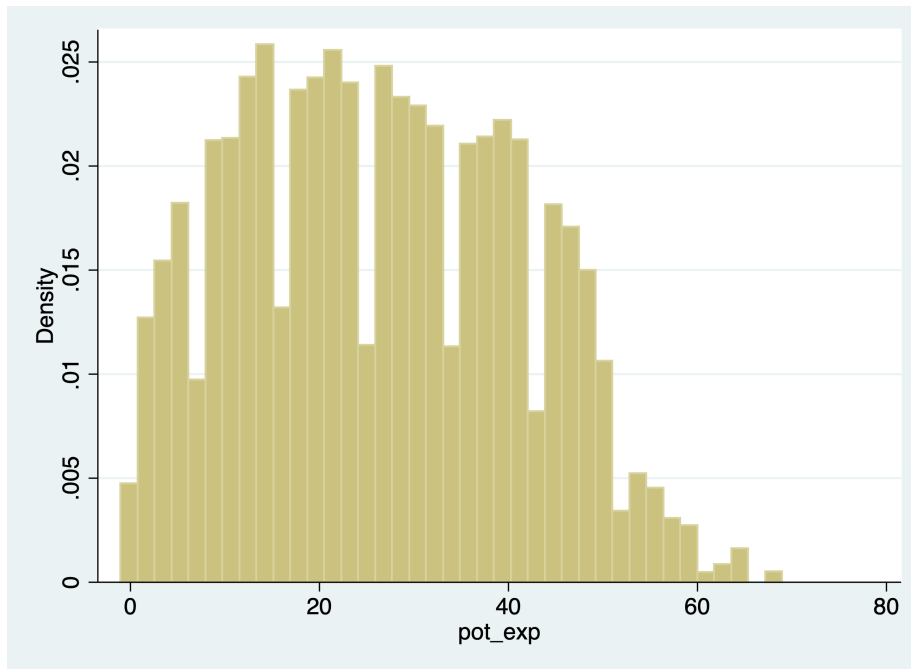


Figure 1.1: Potential Experience

(28,617 missing values generated)

Variable	Obs	Mean	Std. dev.	Min	Max
-----+-----					
pot_exp2	93,635	1269.006	1345.314	0	4761

Notice! Our age variable, prtage, is missing when the value is -1. The data dictionary provides us the information that -1 is a holder for missing data. Please note that when missing values are “”, we will see missings in an outlier check! Why? Because missing values are considered **very large values**. This is important because when creating binary or categorical variables through qualifiers, make sure you top code your qualifiers.

For example, if we are creating a binary on part-time vs full-time. We need to top code so we don't count missing hours as full-time workers,

```
gen fulltime if hours > 35 & hours < 999999,
```

Alternatively, you can use the missing function.

```
gen fulltime if hours > 35 & !missing(hours)
```

Let's generate weekly earnings and plot the data. The CPS data dictionary tells us that pternwa is the weekly earnings, but it needs to be divided by 100 to get two decimal places.

```
gen earnings=pternwa/100
histogram earnings if prerelg==1, title(Weekly Earnings)
graph export "jul_25_weekly_earnings.png", replace
```

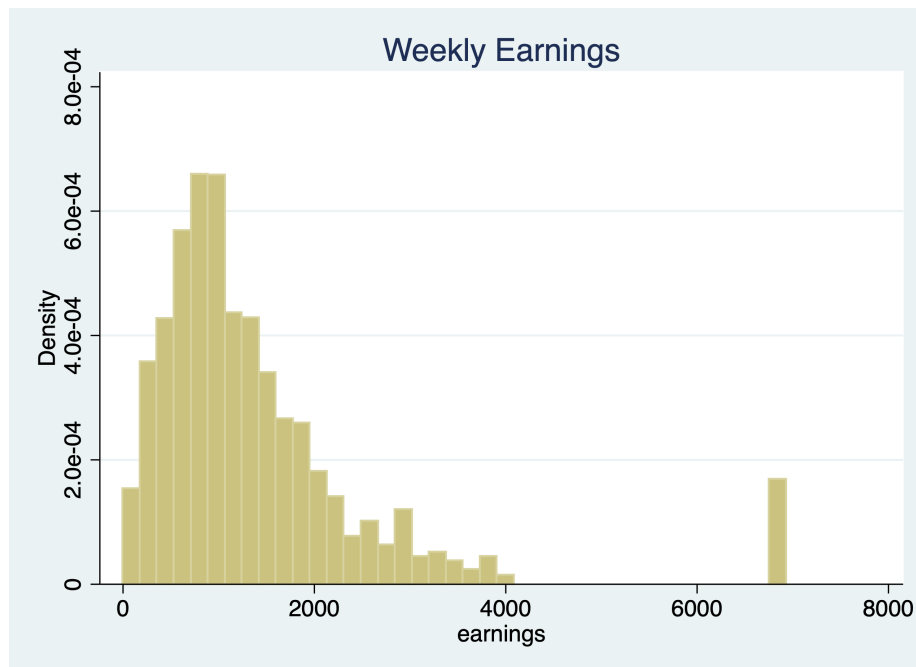


Figure 1.2: Weekly Earnings

Another helpful trick is the before and after options. It might be helpful to have the newly created variable next to a similar variable, so let's drop our variable and generate it again with the after option

```
drop weekwage
gen hrlyearn = pternwa/pehract1, after(pternwa)
drop hrlyearn
gen hrlyearn = pternwa/pehract1, before(pternwa)
```

1.1.2 Replace

We use our *replace* command to modify a variable that has already been created. Similar to generate, you probably already have experience with replace, but we'll go over some useful tips.

Qualifiers will be an essential part of your syntax when using the *replace* command.

Let's generate a variable called marital status and look at married and never-married variables.

```
quietly cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
quietly use wws, clear
tabulate married nevermarried
```

married	Woman never been married		Total
	0	1	
0	570	234	804
1	1,440	2	1,442
Total	2,010	236	2,246

We'll generate a new variable called maritalstatus.

```
gen maritalstatus = .
```

I recommend generating a new categorical variable as missing instead of zero, because this prevents missing variables from being categorized as 0 when applicable.

We generally use qualifiers "=", ">", "<", ">=", "<=", "!" when working with replace. Let's replace the marital status variable with married and nevermarried as qualifiers

```
replace maritalstatus = 1 if married==0 & nevermarried==1
replace maritalstatus = 2 if married==1 & nevermarried==0
replace maritalstatus = 3 if married==0 & nevermarried==0
```

It's always a good idea to double check the variables against the variables they were created from to make sure your categories are what they are supposed to be.

```

tabulate maritalstatus
table maritalstatus married nevermarried

```

maritalstat	us	Freq.	Percent	Cum.
1		234	10.43	10.43
2		1,440	64.17	74.60
3		570	25.40	100.00
Total		2,244	100.00	

		Woman never been married		
		0	1	Total
maritalstatus				
1				
married				
0			234	234
Total			234	234
2				
married				
1		1,440		1,440
Total		1,440		1,440
3				
married				
0		570		570
Total		570		570
Total				
married				
0		570	234	804
1		1,440		1,440
Total		2,010	234	2,244

What will be helpful is to label our values, but we'll cover this next week.

We can also use the *replace* command with a continuous variable in the qualifier

Let's create a new variable called *over40hours* which will be categorized as 1 if it is over 40 hours, and 0 if it is 40 or under.

```
generate over40hours = .
replace over40hours = 0 if hours <= 40
```

We need to make sure we don't include missing values so we need an additional qualifier besides `hours > 40`. We need to add `!missing(hours)`.

```
replace over40hours = 1 if hours > 40 & !missing(hours)
```

Now let's see the results.

```
tab over40hours
tab over40hours, sum(hours)
```

over40hours		Freq.	Percent	Cum.
0		1,852	82.60	82.60
1		390	17.40	100.00
Total		2,242	100.00	

Summary of usual hours worked				
over40hours		Mean	Std. dev.	Freq.
0		34.50054	9.0449086	1,852
1		50.123077	6.6961395	390
Total		37.218109	10.509135	2,242

1.2 Numeric expressions and functions

Our standard numeric expressions 1. addition +, 2. subtraction -, 3. multiplication *, 4. division /, 5. exponentiation ^, and 6. parentheses () for changing order of operation.

We also have some very useful numeric functions built into Stata:

The `int()` function removes any decimals.

The `round()` function rounds a number to the desired decimal place. Please note that this is different from Excel!!!

The `round(x,y)` rounds the numbers to the specified nearest value. Where x is your numeric and y is the nearest value you wish to round to.

For example: 1. if $y = 1$, $\text{round}(-5.2,1)=-5$ and $\text{round}(4.5)=5$ 2. if $y=.1$, $\text{round}(10.16,.1)=10.2$ and $\text{round}(34.1345,.1)=34.1$ 3. if $y=10$, $\text{round}(313.34,10)=310$ and $\text{round}(4.52,10)=0$

Note: if y is missing, then it will round to the nearest integer $\text{round}(10.16)=10$

The $\ln()$ function is our natural log function which we use a lot for transforming variables for elasticity estimates.

The $\log10()$ function is our logarithm base-10 function.

The $\text{sqrt}()$ function is our square root function

```
display int(10.65)
display round(10.65,1)
display ln(10.65)
display log10(10.65)
display sqrt(10.65)
```

10

11

2.3655599

1.0273496

3.2634338

Please note $\text{int}(10.65)$ returns 10, while $\text{round}(10.65,1)$ returns 11.

1.2.1 Random Numbers and setting seeds

There are several random number generating functions in Stata.

$\text{runiform}()$ is a random uniform distribution number generator has a distribution between 0 and 1.

$\text{rnormal}(m, sd)$ is a random normal distribution number generator without arguments it will assume mean = 0 and sd = 1.

$\text{rchi2}(df)$ is a random chi-squared distribution number generator where df is the degrees of freedom that need to be specified

Let's look at some examples.

Setting seeds is important if you want someone to replicate your results, since a set seed will generate the same random number each time it is run.

```

*Set our seed
set seed 12345

*Random uniform distribution
gen runiformtest = runiform()
summarize runiformtest

*Random normal distribution
gen rnormaltest = rnormal()
gen rnormaltest2 = rnormal(100,15)
summarize rnormaltest rnormaltest2

*Random chi-squared distribution
gen rchi2test = rchi2(5)
summarize rchi2test

```

Variable	Obs	Mean	Std. dev.	Min	Max
runiformtest	2,246	.4924636	.2875376	.0007583	.9985868

Variable	Obs	Mean	Std. dev.	Min	Max
rnormaltest	2,246	.018436	.9857847	-3.389102	3.372524
rnormaltest2	2,246	99.70718	14.77732	47.81652	154.2297

Variable	Obs	Mean	Std. dev.	Min	Max
rchi2test	2,246	5.063639	3.322998	.1102785	29.2429

1.3 Strings

Working with string can be a pain, but there are some very helpful functions that will get your task completed.

```
help string functions
```

1.3.1 String expressions and functions

Let's get some new data to work with strings and their functions.

We'll format the names with left-justification 17 characters long.

```
quietly cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use "authors.dta", clear
format name %-17s
list name
```

```
      | name |
      |-----|
1. | Ruth Canaale |
2. | Y. Don Uflossmore |
3. | thích nhất hạnh |
4. | J. Carreño Quiñones |
5. | Ô Knausgård |
      |-----|
6. | Don b Iteme |
7. | isaac O'yerbreath |
8. | Mike avity |
9. | ÉMILE ZOLA |
10. | i William Crown |
      |-----|
11. | Ott W. Onthurt |
12. | Olive Tu'Drill |
13. | björk guðmundsdóttir |
      +-----+
```

Notice white space(s) in front of some names and some names are not in the proper format (lowercase for initial letter instead of upper).

To fix these, we have a three string functions to work with.

The *ustrtitle()* function is the Unicode function to convert strings to title cases, which means that the first letter is always upper and all others are lower case for each word.

The *ustrlower()* is the Unicode function to convert strings to lower case.

The *ustrupper()* is the Unicode function to convert strings to upper case.

Please note that there are string function that do something similar but in ASCII. Our names are not in ASCII currently, so please note that not all string characters come in ASCII, but may come in Unicode: 1. *strproper()* 2. *strlower()* 3. *strupper()*

We'll generate some names and format our strings to 23 length and left-justified

```
generate name2 = ustrtitle(name)
generate lowname = ustrlower(name)
generate upname = ustrupper(name)
```



```
format name2 lowname upname %-23s
list name2 lowname upname
```

	name2	lowname	upname
1.	Ruth Canaale	ruth canaale	RUTH CANAALE
2.	Y. Don Uflossmore	y. don uflossmore	Y. DON UFLOSSMORE
3.	Thích Nhất Hạnh	thích nhất hạnh	THÍCH NHẤT HẠNH
4.	J. Carreño Quiñones	j. carreño quiñones	J. CARREÑO QUIÑONES
5.	Ô Knausgård	ô knausgård	Ô KNAUSGÅRD
6.	Don B Iteme	don b iteme	DON B ITEME
7.	Isaac O'yerbreath	isaac o'yerbreath	ISAAC O'YERBREATH
8.	Mike Avity	mike avity	MIKE AVITY
9.	Émile Zola	émile zola	ÉMILE ZOLA
10.	I William Crown	i william crown	I WILLIAM CROWN
11.	Ott W. Onthurt	ott w. onthurt	OTT W. ONTHURT
12.	Olive Tu'drill	olive tu'drill	OLIVE TU'DRILL
13.	Björk Guðmundsdóttir	björk guðmundsdóttir	BJÖRK GUÐMUNDSDÓTTIR

We still need to get rid of leading white spaces in front of the name. The `ustrltrim()` will get rid of leading blanks.

```
generate name3 = ustrltrim(name2)
format name2 name3 %-17s
list name name2 name3
```

	name	name2	name3
1.	Ruth Canaale	Ruth Canaale	Ruth Canaale
2.	Y. Don Uflossmore	Y. Don Uflossmore	Y. Don Uflossmore
3.	thích nhất hạnh	Thích Nhất Hạnh	Thích Nhất Hạnh
4.	J. Carreño Quiñones	J. Carreño Quiñones	J. Carreño Quiñones
5.	Ô Knausgård	Ô Knausgård	Ô Knausgård
6.	Don b Iteme	Don B Iteme	Don B Iteme
7.	isaac O'yerbreath	Isaac O'yerbreath	Isaac O'yerbreath
8.	Mike avity	Mike Avity	Mike Avity
9.	ÉMILE ZOLA	Émile Zola	Émile Zola
10.	i William Crown	I William Crown	I William Crown
11.	Ott W. Onthurt	Ott W. Onthurt	Ott W. Onthurt

```

12. | Olive Tu'Drill      Olive Tu'drill      Olive Tu'drill      |
13. | björk guðmundsdóttir Björk Guðmundsdóttir Björk Guðmundsdóttir |
    +-----+

```

Let's work with the initials to identify the initial. In small datasets this is easy enough, but for large datasets we'll need to use some functions.

1.3.2 substr functions

One of the more practical string functions is the substr functions:

The *usubstr*(*s*,*n1*,*n2*) is our Unicode substring.

The *substr*(*s*,*n1*,*n2*) is our ASCII substring

Where *s* is the string, *n1* is the starting position, *n2* is the length

```
display substr("abcdef",2,3)
```

```
bcd
```

Let's find names that start with the initial with substr. We'll start at the 2nd position and move 1 character. Let's use ASCII and compare it to Unicode.

```

gen secondchar = substr(name3,2,1)
gen firstinit = (secondchar == " " | secondchar == ".") if !missing(secondchar)

```

We might want to break up a string as well. This is helpful for names, addresses, etc. We can use the strwordcount function.

The *ustrwordcount*(*s*) counts the number of words using word-boundary rules of Unicode strings. Where *s* is the string input.

```

generate namecount = ustrwordcount(name3)
list name3 namecount

```

```

      | name3                      nameco~t |
      |-----|
1.    | Ruth Canaale                2 |
2.    | Y. Don Uflossmore           4 |
3.    | Thích Nhất Hạnh              3 |
4.    | J. Carreño Quiñones         4 |
5.    | Ô Knausgård                 2 |
      |-----|
6.    | Don B Iteme                 3 |

```

```

7. | Isaac O'yerbreath          2 |
8. | Mike Avity                 2 |
9. | Émile Zola                 2 |
10. | I William Crown           3 |
    |-----|
11. | Ott W. Onthurt             4 |
12. | Olive Tu'drill            2 |
13. | Björk Guðmundsdóttir      2 |
    +-----+

```

Notice that there are three authors have a word count of 4 instead of 3 when it should be three.

To extract the words after word count, we can use the `ustrword()` command. `ustrword(s,n)` returns the word in the string depending upon n. **Note:** a positive n returns the nth word from the left, while a -n returns the nth word from the right.

```

generate uname1=ustrword(name3,1)
generate uname2=ustrword(name3,2)
generate uname3=ustrword(name3,3)
generate uname4=ustrword(name3,4)
list name3 uname1 uname2 uname3 uname4 namecount

```

(7 missing values generated)

(10 missing values generated)

```

+-----+
| name3          uname1      uname2      uname3      uname4      namecount |
+-----+
1. | Ruth Canaale      Ruth      Canaale          2 |
2. | Y. Don Uflossmore Y          .      Don      Uflossmore      4 |
3. | Thích Nhất Hạnh    Thích      Nhất      Hạnh          3 |
4. | J. Carreño Quiñones J          .      Carreño      Quiñones      4 |
5. | Ô Knausgård       Ô          Knausgård        2 |
    +-----+
6. | Don B Iteme       Don          B      Iteme          3 |
7. | Isaac O'yerbreath Isaac      O'yerbreath        2 |
8. | Mike Avity        Mike          Avity          2 |
9. | Émile Zola        Émile          Zola          2 |
10. | I William Crown   I          William      Crown          3 |
    +-----+
11. | Ott W. Onthurt     Ott          W          .      Onthurt        4 |
12. | Olive Tu'drill    Olive          Tu'drill        2 |
13. | Björk Guðmundsdóttir Björk      Guðmundsdóttir    2 |

```

It seems that *ustrwordcount* counts “.” as separate words, so let’s use another very helpful string function called *subinstr*, which comes in Unicode *usubinstr()* and ASCII *subinstr()*

usubinstr(s1,s2,s3,n) replaces the *n*th occurrence of *s2* in *s1* with *s3*.

subinstr(s1,s2,s3,n) is the same thing but for ASCII.

Where *s1* is our string, *s2* is the string we want to replace, *s3* is the string we want instead, and *n* is the *n*th occurrence of *s2*.

If *n* is missing or implied, then all occurrences of *s2* will be replaced.

```
generate name4 = usubinstr(name3,".",",",.)
replace namecount = ustrwordcount(name4)
list name4 namecount
```

(3 real changes made)

```
+-----+
|               name4   namecount |
+-----+
1. |      Ruth Canaale           2 |
2. |      Y Don Uflossmore       3 |
3. |      Thích Nhất Hạnh         3 |
4. |      J Carreño Quiñones     3 |
5. |      Ô Knausgård            2 |
+-----+
6. |      Don B Iteme            3 |
7. |      Isaac O'yerbreath       2 |
8. |      Mike Avity             2 |
9. |      Émile Zola             2 |
10. |      I William Crown       3 |
+-----+
11. |      Ott W Onthurt          3 |
12. |      Olive Tu'drill        2 |
13. |      Björk Guðmundsdóttir  2 |
+-----+
```

Let’s split the names. We’ll use *ustrword(s,n)* retrieves the *n*th word in string *s*.

```
gen fname = ustrword(name4,1)
gen mname = ustrword(name4,2) if namecount==3
gen lname = ustrword(name4,namecount)
```

```
format fname mname lname %-15s
list name4 fname mname lname
```

(7 missing values generated)

```
+-----+
|          name4    fname   mname    lname      |
+-----+
1. |      Ruth Canaale  Ruth     Canaale      |
2. |    Y Don Uflossmore Y      Don     Uflossmore   |
3. |    Thích Nhất Hạnh  Thích   Nhất     Hạnh        |
4. |  J Carreño Quiñones J      Carreño  Quiñones    |
5. |      Ô Knausgård   Ô        Knausgård  |
+-----+
6. |      Don B Iteme   Don     B      Iteme        |
7. |    Isaac O'yerbreath Isaac    O'yerbreath  |
8. |      Mike Avity    Mike     Avity        |
9. |      Émile Zola    Émile    Zola        |
10. |  I William Crown  I      William  Crown        |
+-----+
11. |    Ott W Onthurt   Ott     W      Onthurt    |
12. |    Olive Tu'drill  Olive    Tu'drill   |
13. | Björk Guðmundsdóttir Björk    Guðmundsdóttir |
+-----+
```

Some names have the middle initial first, so let's rearrange. The middle initial will only have a string length of one, so we need our string length functions. `ustrlen(s)` returns the length of the string.

`strlen(s)` is the same as above but for ASCII

Let's add the period back to the middle initial if the string length is 1.

```
replace fname=fname + "." if ustrlen(fname)==1
replace mname=mname + "." if ustrlen(mname)==1
list fname mname
```

(4 real changes made)

(2 real changes made)

```
+-----+
| fname   mname   |
+-----+
1. | Ruth      |
```

```

2. | Y.      Don      |
3. | Thích  Nhất      |
4. | J.      Carreño  |
5. | Ô.      |
   |-----|
6. | Don    B.      |
7. | Isaac  |
8. | Mike   |
9. | Émile  |
10. | I.     William |
   |-----|
11. | Ott    W.      |
12. | Olive  |
13. | Björk  |
   +-----+

```

Let's make a new variable that correctly arranges the parts of the name.

```

gen mlen = strlen(mname)
gen flen = strlen(fname)
gen fullname = fname + " " + lname if namecount == 2
replace fullname = fname + " " + mname + " " + lname if namecount==3 & mlen > 2
replace fullname = fname + " " + mname + " " + lname if namecount==3 & mlen==2
replace fullname = mname + " " + fname + " " + lname if namecount==3 & flen==2
list fullname fname mname lname

```

(6 missing values generated)

(4 real changes made)

(2 real changes made)

(3 real changes made)

```

+-----+
|          fullname   fname   mname   lname          |
+-----+
1. |      Ruth Canaale   Ruth      Canaale          |
2. |    Don Y. Uflossmore Y.      Don    Uflossmore          |
3. |    Thích Nhất Hạnh   Thích    Nhất    Hạnh          |
4. | Carreño J. Quiñones  J.      Carreño Quiñones          |
5. |      Ô. Knausgård   Ô.      Knausgård          |
   +-----+
6. |    Don B. Iteme     Don      B.      Iteme          |
7. | Isaac O'yerbreath   Isaac      O'yerbreath          |

```

```

8. |           Mike Avity   Mike           Avity           |
9. |           Émile Zola  Émile           Zola           |
10. |      William I. Crown  I.      William  Crown           |
    |-----|
11. |      Ott W. Onthurt   Ott      W.      Onthurt           |
12. |      Olive Tu'drill  Olive           Tu'drill           |
13. | Björk Guðmundsdóttir Björk           Guðmundsdóttir |
    +-----+

```

If you are brave enough, learning regular express can pay off in the long run. Stata has string functions with regular express, such as finding particular sets of strings with *regexm()*. Regular expressions are a pain to work with, but they are very powerful.

Chapter 2

Labeling Data

Mitchell's 5th chapter has a bunch of useful information, but the most important parts are *label define* (and its two options of *add* and *modify*), *label values*, and time formatting.

2.1 Describing datasets

We have seen the *describe* command before, but it is a very useful command to being working with data. It provides the variable name, storage type, display format, value label, and variable label, Let's get some data on the survey of graduate students.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"  
use "survey7.dta", clear  
describe
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell
```

```
(Survey of graduate students)
```

```
Contains data from survey7.dta
```

```
Observations:      8
```

```
Variables:         11
```

```
Survey of graduate students
```

```
5 May 2020 14:37
```

```
(_dta has notes)
```

Variable name	Storage type	Display format	Value label	Variable label
------------------	-----------------	-------------------	----------------	----------------

```

-----
id          float   %9.0g          Unique identification variable
STUDENTVARS float   %9.0g          STUDENT VARIABLES =====
gender      float   %9.0g          mf      Gender of student
race        float   %19.0g         racelab * Race of student
bday        float   %td..          Date of birth of student
income      float   %11.1fc        Income of student
havechild   float   %18.0g         havelab * Given birth to a child?
KIDVARS     float   %9.0g          KID VARIABLES =====
kidname     str10   %-10s          Name of child
ksex        float   %15.0g         mfkid  * Sex of child
kday        float   %td..          Date of birth of child
                                     * indicated variables have notes
-----

```

Sorted by:

We also have a short option, but it just contain general information.

```
describe, short
```

```

Contains data from survey7.dta
  Observations:      8          Survey of graduate students
    Variables:      11          5 May 2020 14:37
Sorted by:

```

We can subset the variables we want to describe if we want

```
describe id gender race
```

```

-----
Variable      Storage   Display   Value
   name        type     format    label      Variable label
-----
id            float    %9.0g          Unique identification variable
gender        float    %9.0g          mf      Gender of student
race          float    %19.0g         racelab * Race of student

```

Finally, the command *codebook* provides a deep dive into your dataset. This is very useful for looking at the value labels. We only see the value label name in the *describe* command, but the *codebook* command provides more information, such as type of variable, label name, range of values, unique values, missing, value labels, missing value labels (if any).

id	Unique identification variable
----	--------------------------------

STUDENTVARS STUDENT VARIABLES =====

gender Gender of student

```

Type: Numeric (float)
Label: mf

Range: [1,2]
Units: 1
Unique values: 2
Missing .. 0/8

Tabulation: Freq.    Numeric  Label
              3          1  Male
              5          2  Female

```

race

Race

Type: Numeric (float)

Label: racelab

Range: [1,5]
 Unique values: 5

Units: 1
 Missing .: 0/8

Tabulation: Freq.	Numeric	Label
2	1	White
2	2	Asian
2	3	Hispanic
1	4	African American
1	5	Other

bday

Date of birth

Type: Numeric daily date (float)

Range: [389,7935]
 Or equivalently: [24jan1961,22sep1981]
 Unique values: 8

Units: 1
 Units: days
 Missing .: 0/8

Tabulation: Freq.	Value
1	389 24jan1961
1	3027 15apr1968
1	4160 23may1971
1	4924 25jun1973
1	5036 15oct1973
1	6059 03aug1976
1	6391 01jul1977
1	7935 22sep1981

income

Income

Type: Numeric (float)

Range: [545.23,1284354.5]
 Unique values: 8

Units: .01
 Missing .: 0/8

```

Tabulation: Freq.  Value
              1  545.22998
              1  4500.9199
              1  10500.93
              1  45234.129
              1  109452.11
              1  120102.32
              1  124313.45
              1  1284354.5

```

havechild

Given birth to a child

```

Type: Numeric (float)
Label: havelab

```

```

Range: [0,1]
Unique values: 2
Unique mv codes: 1
Units: 1
Missing .: 0/8
Missing .*: 3/8

```

```

Tabulation: Freq.  Numeric  Label
              1         0  Dont Have Child
              4         1  Have Child
              3         .n  NA

```

KIDVARS

KID VARIABLES =====

```

Type: Numeric (float)
Range: [.,.]
Unique values: 0
Units: .
Missing .: 8/8

```

```

Tabulation: Freq.  Value
              8  .

```

kidname

Name of child

```

Type: String (str10), but longest is str9
Unique values: 5
Missing "": 0/8

```

```

Tabulation: Freq.  Value
              4  ""
              1  "Catherine"
              1  "Robin"
              1  "Sally"
              1  "Samuell"

```

Warning: Variable has leading and trailing blanks.

ksex

```

Type: Numeric (float)
Label: mfkid

Range: [1,2]
Unique values: 2
Unique mv codes: 2

Units: 1
Missing .: 0/8
Missing .*: 5/8

```

```

Tabulation: Freq.  Numeric  Label
              1      1  Male
              2      2  Female
              4      .n  NA
              1      .u  Unknown

```

kbday

Date of birth

```

Type: Numeric daily date (float)

Range: [12888,15932]
Or equivalently: [15apr1995,15aug2003]
Unique values: 4

Units: 1
Units: days
Missing .: 4/8

```

```

Tabulation: Freq.  Value
              1  12888  15apr1995
              1  14019  20may1998
              1  14256  12jan1999
              1  15932  15aug2003
              4  .      .

```

We can go by variables.

```
codebook race
```

```
race
```

```
Race of student
```

```
-----
Type: Numeric (float)
Label: racelab

Range: [1,5]
Unique values: 5

Units: 1
Missing .: 0/8

Tabulation: Freq.  Numeric  Label
              2           1  White
              2           2  Asian
              2           3  Hispanic
              1           4  African American
              1           5  Other
```

We can go by variables and notes

```
codebook havechild, notes
```

```
havechild
```

```
Given birth to a child
```

```
-----
Type: Numeric (float)
Label: havelab

Range: [0,1]
Unique values: 2
Unique mv codes: 1

Units: 1
Missing .: 0/8
Missing .*: 3/8

Tabulation: Freq.  Numeric  Label
              1           0  Dont Have Child
              4           1  Have Child
              3          .n  NA
```

havechild:

1. This variable measures whether a woman has given birth to a child, not just whether she is parent.
2. The .n (NA) missing code is used for males, because they cannot bear children.
3. The .u (Unknown) missing code for a female indicating it is unknown if she has a child.

We can look at the variable and missing value labels with the option *mv*. I recommend that you don't label the missing values unless it is absolutely nec-

essary. Different types of missing values besides “.” cause problems down the road, especially with the *marginsplot* command.

```
codebook ksex, mv
```

```
ksex
```

```

                                Type: Numeric (float)
                                Label: mfkid

                                Range: [1,2]
                                Unique values: 2
                                Unique mv codes: 2

                                Units: 1
                                Missing .: 0/8
                                Missing .*: 5/8

                                Tabulation: Freq.   Numeric   Label
                                                1           1   Male
                                                2           2   Female
                                                4           .n   NA
                                                1           .u   Unknown

                                Missing values:      havechild==mv --> ksex==mv
                                                    kbday==mv --> ksex==mv

```

If you are interested in the different languages labels it is on page 112.

The *lookfor* command will return all variables with the search word. This is a bit redundant, since this is available in the variable window. But, it provides more space to look.

```
lookfor birth
```

Variable name	Storage type	Display format	Value label	Variable label
bday	float	%td..		Date of birth of student
havechild	float	%18.0g	havelab *	Given birth to a child?
kbday	float	%td..		Date of birth of child

We can also search for the notes by the search word

```
notes search birth
```

```
havechild:
```

1. This variable measures whether a woman has given birth to a child, not just whether she is a parent.

We can see the formats of the variables as well

```
list income bday
describe income bday
```

```

      |      income      bday |
      |-----|
1. |    10,500.9    01/24/61 |
2. |    45,234.1    04/15/68 |
3. |  1,284,354.5    05/23/71 |
4. |   124,313.5    06/25/73 |
5. |   120,102.3    09/22/81 |
      |-----|
6. |      545.2    10/15/73 |
7. |   109,452.1    07/01/77 |
8. |    4,500.9    08/03/76 |
+-----+
```

Variable name	Storage type	Display format	Value label	Variable label
income	float	%11.1fc		Income of student
bday	float	%td..		Date of birth of student

We can see that the format for income is %11.1fc and the format for bday is %td

2.2 Labeling variables

Labeling the variables is a very helpful shortcut to describe what the variable contain without having to go back to the data dictionary. Sometimes we want a short and concise label if we are exporting labels to regression tables, or sometimes we want longer variable labels to give us context of the variable.

Let's get some data on graduate students and use the *describe* command.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use "survey1.dta", clear
describe
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell
```

Contains data from survey1.dta

Observations: 8
Variables: 9 1 Jan 2010 12:13

Variable name	Storage type	Display format	Value label	Variable label
id	float	%9.0g		
gender	float	%9.0g		
race	float	%9.0g		
havechild	float	%9.0g		
ksex	float	%9.0g		
bdays	str10	%10s		
income	float	%9.0g		
kbdays	str10	%10s		
kidname	str10	%10s		

Sorted by:

We have no variable labels, so we will need to provide some so future users have an understand what the data are. We will use the *label variable* command to describe the variable.

```
label variable id "Identification variable"
label variable gender "Gender of the student"
label variable race "Race of the student"
label variable havechild "Given birth to a child"
label variable ksex "Sex of child"
label variable bday "Birthday of student"
label variable income "Income of student"
label variable kbdays "Birthday of child"
label variable kidname "Name of child"
describe
```

Contains data from survey1.dta

Observations: 8
Variables: 9 1 Jan 2010 12:13

Variable name	Storage type	Display format	Value label	Variable label
id	float	%9.0g		Identification variable
gender	float	%9.0g		Gender of the student

race	float	%9.0g	Race of the student
havechild	float	%9.0g	Given birth to a child
ksex	float	%9.0g	Sex of child
bdays	str10	%10s	Birthday of student
income	float	%9.0g	Income of student
kbdays	str10	%10s	Birthday of child
kidname	str10	%10s	Name of child

Sorted by:

Note: Dataset has changed since last saved.

We can simply change the variable label with running the command again with the new variable label.

```
label variable id "Unique identification variable"
describe
save survey2, replace
```

Contains data from survey1.dta

Observations: 8

Variables: 9

1 Jan 2010 12:13

Variable name	Storage type	Display format	Value label	Variable label
id	float	%9.0g		Unique identification variable
gender	float	%9.0g		Gender of the student
race	float	%9.0g		Race of the student
havechild	float	%9.0g		Given birth to a child
ksex	float	%9.0g		Sex of child
bdays	str10	%10s		Birthday of student
income	float	%9.0g		Income of student
kbdays	str10	%10s		Birthday of child
kidname	str10	%10s		Name of child

Sorted by:

Note: Dataset has changed since last saved.

file survey2.dta saved

2.3 Labeling values

Labeling values is a very practice way of analyzing data without having to go back to the data dictionary. Labeling values requires two commands:

- Labeling values is a bit different than labeling variables, since we need to modify or replace after a label has been defined. If you try to change a label, then you will get an error unless you use *add*, *modify* or *replace*

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use survey2, clear
codebook
```

id	Unique identificat.
----	---------------------

```

      Range: [1,8]
Unique values: 8
      Units: 1
Missing .: 0/8

```

gender Gender of

```

      Range: [1,2]
Unique values: 2
      Units: 1
Missing .: 0/8

```

```

Tabulation: Freq.  Value
              3   1
              5   2

```

race

Race of the student

Type: Numeric (float)

Range: [1,5]
Unique values: 5

Units: 1
Missing .: 0/8

```

Tabulation: Freq.  Value
              2   1
              2   2
              2   3
              1   4
              1   5

```

havechild

Given birth to a child

Type: Numeric (float)

Range: [0,1]
Unique values: 2
Unique mv codes: 1

Units: 1
Missing .: 0/8
Missing .*: 3/8

```

Tabulation: Freq.  Value
              1   0
              4   1
              3  .n

```

ksex

Sex of child

Type: Numeric (float)

Range: [1,2]
Unique values: 2
Unique mv codes: 2

Units: 1
Missing .: 0/8
Missing .*: 5/8

```

Tabulation: Freq.  Value

```

```

1 1
2 2
4 .n
1 .u

```

bdays

Birthda

Type: String (str10)

Unique values: 8

Missing "": 0/8

```

Tabulation: Freq.  Value
1  "1/24/1961"
1  "10/15/1973"
1  "4/15/1968"
1  "5/23/1971"
1  "6/25/1973"
1  "7/1/1977"
1  "8/3/1976"
1  "9/22/1981"

```

income

Incom

Type: Numeric (float)

Range: [545.23,1284354.5]

Units: .01

Unique values: 8

Missing .: 0/8

```

Tabulation: Freq.  Value
1  545.22998
1  4500.9199
1  10500.93
1  45234.129
1  109452.11
1  120102.32
1  124313.45
1  1284354.5

```

kdays

Birth

Type: String (str10), but longest is str9

Unique values: 5

Missing "": 0/8

Tabulation: Freq.	Value
4	"
1	"1/12/1999"
1	"4/15/1995"
1	"5/20/1998"
1	"8/15/2003"

Warning: Variable has leading and trailing blanks.

kidname

Name of child

Type: String (str10), but longest is str9

Unique values: 5

Missing "": 0/8

Tabulation: Freq.	Value
4	"
1	"Catherine"
1	"Robin"
1	"Sally"
1	"Samuell"

Warning: Variable has leading and trailing blanks.

We have our variable labels from 5.3, but now we need to label the values so replicators can know what the data are without having to reference the data dictionary for every variable. We will use the *label define* command to create a new label.

First we need to define a label with label define

```
label define racelabel 1 "White" 2 "Asian" 3 "Hispanic" 4 "Black"
```

Next we need to label the values of the variable with *label values* command and we'll look at the codebook again.

```
label values race racelabel
codebook race
```

race

Race of the student

```

      Type: Numeric (float)
      Label: racelabel, but 1 nonmissing value is not labeled

      Range: [1,5]
Unique values: 5
      Units: 1
      Missing .: 0/8

      Tabulation: Freq.   Numeric   Label
                   2         1   White
                   2         2   Asian
                   2         3   Hispanic
                   1         4   Black
                   1         5

```

We are still missing a value label for 5, which is Other, so we need to modify our defined label `racel`. If we do not modify our label, we will get an error if we try to label values again. We can use the *add* option in label define.

```
label define racelabel 5 "Other", add
```

If we want to modify an existing label, we can use the *modify* option in label define.

```
label define racelabel 4 "African American", modify
codebook race
```

```

race
Race of

```

```

      Type: Numeric (float)
      Label: racelabel

      Range: [1,5]
Unique values: 5
      Units: 1
      Missing .: 0/8

      Tabulation: Freq.   Numeric   Label
                   2         1   White
                   2         2   Asian
                   2         3   Hispanic
                   1         4   African American
                   1         5   Other

```

Labeling missing is something that I don't recommend, but we'll show an example here.


```

label define mfkid 1 "Male" 2 "Female" .u "Unknown" .n "NA"
label values ksex mfkid
codebook ksex

label define havechildlabel 0 "Don't have a child" 1 "Have a child" .u "Unknown" .n "NA"
label values havechild havechildlabel
codebook havechild

```

ksex Sex of child

```

Type: Numeric (float)
Label: mfkid

Range: [1,2]
Unique values: 2
Unique mv codes: 2

Units: 1
Missing .: 0/8
Missing .*: 5/8

```

Tabulation: Freq.	Numeric	Label
1	1	Male
2	2	Female
4	.n	NA
1	.u	Unknown

havechild Given birth to a child

```

Type: Numeric (float)
Label: havechildlabel

Range: [0,1]
Unique values: 2
Unique mv codes: 1

Units: 1
Missing .: 0/8
Missing .*: 3/8

```

Tabulation: Freq.	Numeric	Label
1	0	Don't have a child
4	1	Have a child
3	.n	NA

We can look at our label list to see what we have define so far. We can use the *label list* command to see our labels.

```
label list
```

```
havechildlabel:
    0 Don't have a child
    1 Have a child
.n NA
.u Unknown
mfkid:
    1 Male
    2 Female
.n NA
.u Unknown
racelabel:
    1 White
    2 Asian
    3 Hispanic
    4 African American
    5 Other
```

The *numlabel* command is an interesting command. It takes the guess work out of knowing the numeric value of the category by appending the numeric value with the label value.

```
numlabel racelabel, add
label list racelabel
tabulate race
```

```
racelabel:
    1 1. White
    2 2. Asian
    3 3. Hispanic
    4 4. African American
    5 5. Other
```

Race of the student	Freq.	Percent	Cum.
1	2	25.00	25.00
2	2	25.00	50.00
3	2	25.00	75.00
4	1	12.50	87.50
5	1	12.50	100.00
Total	8	100.00	

And, if we don't like it or don't need it any more, we can remove the numeric values with the *remove* option.

```
numlabel racelabel, remove
tabulate race
```

Race of the student		Freq.	Percent	Cum.
1		2	25.00	25.00
2		2	25.00	50.00
3		2	25.00	75.00
4		1	12.50	87.50
5		1	12.50	100.00
Total		8	100.00	

We can add additional characters with *numlabel* as well, such as “#=” or “#)” with the *mask* option

```
numlabel racelabel, add mask("#) ")
tabulate race
```

Race of the student		Freq.	Percent	Cum.
1		2	25.00	25.00
2		2	25.00	50.00
3		2	25.00	75.00
4		1	12.50	87.50
5		1	12.50	100.00
Total		8	100.00	

We can remove the mask with *remove* plus the *mask* option

```
numlabel racelabel, remove mask("#) ")
tabulate race
```

Race of the student		Freq.	Percent	Cum.
1		2	25.00	25.00
2		2	25.00	50.00

3	2	25.00	75.00
4	1	12.50	87.50
5	1	12.50	100.00
Total	8	100.00	

```
save survey3, replace
```

2.4 Labeling utilities

Stata has label utilities to manage the labels defined. The first one is *label dir* to see the labels names available in a quick and more concise way than using the *codebook* command.

For me, I think that *label list* will be your most useful command here.

Quick check of your label directory

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use survey3, clear
label dir
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell
```

```
mfkid
havechildlabel
racelabel
```

The *label list* command gives a more comprehensive view of your labels that includes the value labels associated with the value label name

```
label list
```

```
mfkid:
      1 Male
      2 Female
      .n NA
      .u Unknown
havechildlabel:
      0 Don't have a child
      1 Have a child
      .n NA
```

```

        .u Unknown
racelabel:
    1 White
    2 Asian
    3 Hispanic
    4 African American
    5 Other

```

The *label save* command will save your labels into a do file for future use. Our do file name is stated after the using statement.

```
label save havechildlabel racelabel using surveylabs, replace
type surveylabs.do
```

```

label define havechildlabel 0 `Don't have a child', modify
label define havechildlabel 1 `Have a child', modify
label define havechildlabel .n `NA', modify
label define havechildlabel .u `Unknown', modify
label define racelabel 1 `White', modify
label define racelabel 2 `Asian', modify
label define racelabel 3 `Hispanic', modify
label define racelabel 4 `African American', modify
label define racelabel 5 `Other', modify

```

The *labelbook* command provides information similar to *codebook* but only for the labels that are defined.

```
labelbook
labelbook racelabel
```

Value label havechildlabel

```

-----
Values                                Labels
Range:  [0,1]                        String length:  [2,18]
      N:   4                          Unique at full length:  yes
      Gaps: no                        Unique at length 12:  yes
Missing .*: 2                        Null string:  no
                                      Leading/trailing blanks:  no
                                      Numeric -> numeric:  no

Definition
      0   Don't have a child
      1   Have a child
      .n   NA

```

.u Unknown

Variables: havechild

Value label mfkid

Values	Labels
Range: [1,2]	String length: [2,7]
N: 4	Unique at full length: yes
Gaps: no	Unique at length 12: yes
Missing .*: 2	Null string: no
	Leading/trailing blanks: no
	Numeric -> numeric: no
Definition	
1 Male	
2 Female	
.n NA	
.u Unknown	

Variables: ksex

Value label racelabel

Values	Labels
Range: [1,5]	String length: [5,16]
N: 5	Unique at full length: yes
Gaps: no	Unique at length 12: yes
Missing .*: 0	Null string: no
	Leading/trailing blanks: no
	Numeric -> numeric: no
Definition	
1 White	
2 Asian	
3 Hispanic	
4 African American	
5 Other	

Variables: race

Value label racelabel

Values	Labels
Range: [1,5]	String length: [5,16]
N: 5	Unique at full length: yes
Gaps: no	Unique at length 12: yes
Missing .*: 0	Null string: no
	Leading/trailing blanks: no
	Numeric -> numeric: no

Definition

1	White
2	Asian
3	Hispanic
4	African American
5	Other

Variables: race

The *problem* option for *labelbook* provides information to alert the users of any problems

```
labelbook, problem
```

```
no potential problems in dataset survey3.dta
```

We can have a more detailed look with the *detail* and *problem* options

```
labelbook racelabel, problem detail
```

Value label racelabel

Values	Labels
Range: [1,5]	String length: [5,16]
N: 5	Unique at full length: yes
Gaps: no	Unique at length 12: yes
Missing .*: 0	Null string: no
	Leading/trailing blanks: no
	Numeric -> numeric: no

Definition

1	White
---	-------

```

2   Asian
3   Hispanic
4   African American
5   Other

```

```
Variables:  race
```

```
no potential problems in dataset survey3.dta
```

2.5 Labeling variables and values in different languages

We will not be covering this, but if you are interested, please review pages 127-132.

2.6 Using Notes

The *note* command can be helpful for future users or for replicators. If you use the *note* command without specifying the variable, then it is a general note that will show up under the `__dta` note. If you add a variable in front of the *note* command, like `note var1:`, then you will add a note to the variable

Let's add some general notes.

```
note: This was based on the dataset called survey1.txt
```

Adding *TS* to the end adds a timestamp, which is a nice feature.

```
note: The missing values for havechild and chldage were coded using -1 and -2 but were
```

Let's call our notes with the *notes* command.

```
notes
```

```
__dta:
```

1. This was based on the dataset called survey1.txt
2. The missing values for havechild and chldage were coded using -1 and -2 but were
to .n and .u 30 Aug 2026 10:39

Let's add some notes to our variables


```
note race: The other category includes people who specified multiple races
note race: This is another note
note race: This is a third note
```

We can just call a particular variable for notes

```
notes race
```

We can just call a particular variable for notes

```
race:
  1. The other category includes people who specified multiple races
  2. This is another note
  3. This is a third note
```

Let say we added an unhelpful note, then we can drop it with the *notes drop* command and we want to drop the second note.

```
notes drop race in 2
notes race
```

```
(1 note dropped)
```

```
race:
  1. The other category includes people who specified multiple races
  3. This is a third note
```

Notice that we have a gap in the sequence of numbering. We can fix that with the *notes renumber* command.

```
notes renumber race
notes race
```

```
race:
  1. The other category includes people who specified multiple races
  2. This is a third note
```

We can also search notes with the notes search “string” command.

```
notes search .u
```

```
_dta:
  2. The missing values for havechild and chldage were coded using -1 and -2 but were converted
    to .n and .u 30 Aug 2026 10:39
```


Chapter 3

Exercises

3.1 Current Population Survey

We will work with the raw data public use micro data from the Current Population Survey. The July 2026 data can be found here: <https://www.census.gov/data/datasets/time-series/demo/cps/cps-basic.html>

For creating variables and labeling data, we will need the data dictionary: https://www2.census.gov/programs-surveys/cps/datasets/2026/basic/2026_Basic_CPS_Public_Use_Record_Layout_plus_IO_Code_list.txt

Steps

1. Import the csv file
2. Generate a binary variable called *female*, based on *pesex*
3. Label the variable “Individual is female”
4. Label the values 0 “Male” and 1 “Female”
5. Tabulate the results to make sure all values were correctly categorized

Steps

1. Generate a nominal categorical variable called *race* based on *ptdtrace* and *pehspnon*.
2. Label the variable “Individual’s race/ethnicity”
3. Label the values 1 “Non-Hispanic White” 2 “Non-Hispanic Black” 3 “Non-Hispanic AIAN” 4 “Non-Hispanic Asian” 5 “Non-Hispanic NHOPI” 6 “Latino/a” 7 “Non-Hispanic Multiracial”
4. Tabulate the results to make sure all values were correctly categorized

Steps

1. Generate a binary variable called *union* based on *peermlab*
2. Label the variable “Union member status”
3. Label the values 0 “Not a union member” 1 “Union member”
4. Tabulate the results to make sure all values were correctly categorized

Steps

1. Generate a continuous variable called earnings
2. Divide the variable by *pternwa* by 100
3. Label the variable “Weekly earnings”
4. Summarize the variable
5. Summarize the variable again but with the qualifier that *prerelg==1*
6. Generate a histogram with the qualifier
7. Compare weekly earnings among race, sex, and union members with the *table* command
8. What is the unweighted mean weekly earnings for a unionized non-Hispanic multiracial female and a non-unionized non-Hispanic multiracial female?

Mitchell, N.M. (2020). *Data Management Using Stata: A Practical Handbook*. (2nd ed.). Stata Press.

UCLA Advanced Research Computing. (2025). *How can I access information stored after I run a command in Stata*. Retrieved from <https://stats.oarc.ucla.edu/stata/faq/how-can-i-access-information-stored-after-i-run-a-command-in-stata-returned-results/>

Wooldridge (2009). *Introductory Econometrics: A Modern Approach*. (4th ed.). South-Western Cengage Learning.